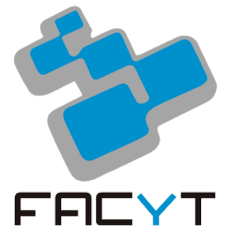




UNIVERSIDAD DE CARABOBO
Facultad Experimental de Ciencias y Tecnología
Departamento de Computación
Asignaturas Electivas
Aprendizaje Automatizado



PYTHON Y NOTEBOOKS: BUENAS PRÁCTICAS

Autor: Prof. Álvaro Espinoza

Febrero, 2026

1. Objetivo

El propósito de estas normas es garantizar que todos los trabajos desarrollados en la asignatura sean:

- Reproducibles
- Claros
- Técnicamente sólidos
- Evaluables de manera objetiva
- Mantenibles por terceros

Un análisis que solo funciona en el computador del autor, que depende del orden en que se ejecutaron las celdas o que no puede entenderse sin explicación verbal adicional, **no cumple el estándar académico del curso**.

El cumplimiento de estas normas formará parte de la evaluación.

2. Organización del Proyecto

El código debe estar estructurado de forma profesional.

2.1 Estructura recomendada

```
project/  
|  
├─ data/  
├─ notebooks/  
├─ src/  
├─ reports/  
└─ requirements.txt
```

- `notebooks/`: exploración, experimentos y explicación.
- `src/`: funciones reutilizables, lógica de negocio, pipelines.

- data/: solo muestras pequeñas (no subir datasets pesados).
- reports/: figuras y resultados finales.

2.2 Código reutilizable

El código repetido no debe permanecer dentro del notebook.

Incorrecto

```
# Todo dentro del notebook
def clean(df):
    df["name"] = df["name"].str.lower()
    return df
```

Correcto

```
# src/preprocessing.py
def clean_names(df):
    cleaned_df = df.copy()
    cleaned_df["name"] = cleaned_df["name"].str.lower()
    return cleaned_df
```

3. Buenas Prácticas de Python

3.1 Imports

Reglas:

- Declararse al inicio.
- Un import por línea.
- No usar import *.
- Orden: estándar → externos → locales.

Incorrecto

```
from pandas import *
import numpy as np, pandas as pd
```

Correcto

```
import os
from pathlib import Path

import numpy as np
import pandas as pd

from src.utils import build_features
```

3.2 Nombres claros y consistentes

Convenciones:

- Variables y funciones: `snake_case`
- Clases: `PascalCase`
- Constantes: `UPPER_CASE`

Incorrecto

```
d = pd.read_csv("data.csv")
t = d[d["a"] > 0]
```

Correcto

```
raw_df = pd.read_csv("data.csv")
filtered_df = raw_df[raw_df["a"] > 0].copy()
```

3.3 Evitar valores “mágicos”

Los parámetros deben declararse explícitamente.

Incorrecto

```
df = df[df["age"] < 120]
model = LogisticRegression(C=0.37)
```

Correcto

```
MAX_AGE = 120
LOGREG_C = 0.37
```

```
df = df[df["age"] < MAX_AGE].copy()
model = LogisticRegression(C=LOGREG_C)
```

3.4 Funciones limpias y sin efectos colaterales

Las funciones no deben modificar objetos originales silenciosamente.

Incorrecto

```
def clean(df):
    df["name"] = df["name"].str.lower()
    return df
```

Correcto

```
def clean_names(df):
    cleaned_df = df.copy()
    cleaned_df["name"] = cleaned_df["name"].str.lower()
    return cleaned_df
```

3.5 Reproducibilidad

Toda operación aleatoria debe incluir semilla fija.

Incorrecto

```
train_test_split(X, y, test_size=0.2)
```

Correcto

```
RANDOM_STATE = 42
```

```
train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=RANDOM_STATE,
    stratify=y
)
```

3.6 Manejo de rutas

Nunca usar rutas locales absolutas.

Incorrecto

```
df = pd.read_csv("C:\\Users\\usuario\\Desktop\\data.csv")
```

Correcto

```
from pathlib import Path
```

```
DATA_DIR = Path("data")  
df = pd.read_csv(DATA_DIR / "data.csv")
```

3.7 Manejo de excepciones

Está prohibido usar except: sin especificar error.

Incorrecto

```
try:  
    process()  
except:  
    pass
```

Correcto

```
try:  
    process()  
except ValueError as exc:  
    raise ValueError("Formato inválido.") from exc
```

3.8 Logging

En scripts largos o pipelines, utilizar logging en lugar de múltiples print().

3.9 Guardado de resultados

Los archivos generados deben tener nombres descriptivos y, cuando corresponda, incluir timestamp o versión.

4. Normas para Jupyter Notebooks

4.1 Reproducibilidad total

El notebook debe ejecutarse correctamente mediante:

Kernel → Restart & Run All

Si falla, no cumple el estándar.

4.2 Organización obligatoria por secciones

1. Configuración
2. Carga de datos
3. Limpieza
4. EDA
5. Feature engineering
6. Modelado
7. Evaluación
8. Conclusiones

Cada sección debe incluir explicación en Markdown.

4.3 No mezclar múltiples tareas en una sola celda

Cada celda debe tener un propósito claro.

4.4 No imprimir datasets completos

Incorrecto

```
df
```

Correcto

```
df.head()  
df.info()  
df.describe()
```

4.5 Visualizaciones obligatoriamente completas

Todo gráfico debe incluir:

- Título
- Etiquetas de ejes
- Unidades si aplica

Incorrecto

```
plt.hist(df["age"])
```

Correcto

```
plt.hist(df["age"], bins=30)
plt.title("Distribution of Age")
plt.xlabel("Age (years)")
plt.ylabel("Count")
```

4.6 Evitar inplace=True (SOLO EN NOTEBOOKS)

Preferir reasignación explícita.

4.7 Bloque de configuración inicial

Todo notebook debe iniciar con:

- Seed
- Paths
- Parámetros globales
- Creación de carpetas necesarias

Ejemplo:

```
from pathlib import Path

RANDOM_STATE = 42
DATA_DIR = Path("data")
FIG_DIR = Path("reports/figures")
FIG_DIR.mkdir(parents=True, exist_ok=True)
```

4.8 Dependencias

No se permite instalar librerías dentro del notebook.

Las dependencias deben declararse en requirements.txt o environment.yml.

5. Normas PEP 8 Obligatorias

5.1 Indentación y longitud de línea

- 4 espacios por nivel
- Máximo recomendado: 79 caracteres por línea

5.2 Espaciado

Incorrecto

```
x= 1+2  
f( x ,y )
```

Correcto

```
x = 1 + 2  
f(x, y)
```

5.3 Comparaciones

Incorrecto

```
if x == None:
```

Correcto

```
if x is None:
```

5.4 Booleanos

Incorrecto

```
if is_valid == True:
```

Correcto

```
if is_valid:
```

5.5 Docstrings

Las funciones reutilizables deben incluir:

- Qué hace
- Qué recibe
- Qué devuelve

Ejemplo:

```
def drop_missing_rows(df):  
    """  
        Remove rows with any missing values.  
  
        Parameters
```

```

-----
df : pandas.DataFrame
    Input dataset.

Returns
-----
pandas.DataFrame
    Dataset without missing rows.
"""
return df.dropna().copy()

```

5.6 Separación entre funciones

Debe haber dos líneas en blanco entre funciones de nivel superior.

5.7 Comentarios

Los comentarios deben explicar el motivo de una decisión, no repetir lo evidente.

6. Checklist Obligatorio de Entrega

Checklist General

- El notebook corre desde cero con Restart & Run All
- No existen rutas locales absolutas
- Todas las operaciones aleatorias tienen seed fija
- No hay import *
- No hay except: vacíos
- Los gráficos tienen título y etiquetas
- No se imprimen datasets completos
- El código repetido fue convertido en funciones
- Las dependencias están documentadas

Checklist de Estilo (PEP 8)

- 4 espacios de indentación
- Variables en snake_case
- Clases en PascalCase
- Constantes en UPPER_CASE
- Espacios correctos alrededor de operadores
- No se comparan booleanos con True/False
- Uso correcto de is None

- Funciones importantes con docstring

7. Consideración Final

El objetivo del curso no es únicamente que el modelo funcione.

El objetivo es formar criterio profesional.

Un buen trabajo en Ciencia de Datos no se evalúa solo por el accuracy, sino por:

- Claridad
- Rigor
- Reproducibilidad
- Calidad del código
- Capacidad de comunicación

Cumplir estas normas no es opcional.

Es parte de la formación técnica del curso.

8. Referencias

Las siguientes referencias respaldan las normas establecidas en este documento y constituyen material recomendado para profundizar en buenas prácticas de programación, análisis reproducible y desarrollo profesional en Ciencia de Datos.

Estándares y estilo de código

- Python Software Foundation. (2001–actualidad). *PEP 8 – Style Guide for Python Code*.
<https://peps.python.org/pep-0008/>
- Van Rossum, G., Warsaw, B., & Coghlan, N. (2001–actualidad). *PEP 257 – Docstring Conventions*.
<https://peps.python.org/pep-0257/>
- Python Software Foundation. *The Python Standard Library Documentation*.
<https://docs.python.org/3/library/>

Ciencia de Datos y buenas prácticas

- McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O'Reilly Media.
(Guía práctica para uso profesional de Pandas y estructuras de datos.)
- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* (3rd ed.). O'Reilly Media.
(Buenas prácticas en modelado y experimentación reproducible.)

- Scikit-learn Developers. *Scikit-learn User Guide*.
https://scikit-learn.org/stable/user_guide.html

Reproducibilidad y análisis científico

- Rule, A., Birmingham, A., Zuniga, C., et al. (2019). *Ten Simple Rules for Writing and Sharing Computational Analyses in Jupyter Notebooks*. PLOS Computational Biology.
<https://doi.org/10.1371/journal.pcbi.1007007>
- Sandve, G. K., et al. (2013). *Ten Simple Rules for Reproducible Computational Research*. PLOS Computational Biology.
<https://doi.org/10.1371/journal.pcbi.1003285>

Ingeniería de Software aplicada a Data Science

- Hunt, A., & Thomas, D. (2019). *The Pragmatic Programmer* (20th Anniversary Edition). Addison-Wesley.
- Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
- Wilson, G., et al. (2014). *Best Practices for Scientific Computing*. PLOS Biology.
<https://doi.org/10.1371/journal.pbio.1001745>